SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 40%

Duration: 1 Hour

QUESTIONS: 3 **Total Marks:30**

Annexure A

Dry Run Evaluation

Code of Conduct:

I M. Santosh (192521007) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

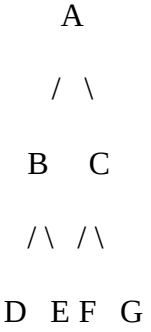
Signature of the student: 

Annexure A

Dry Run Evaluation

1. Question 1 – Breadth-First Search (BFS)

Consider the following graph:



Task: Starting from node **A**, perform a **Breadth-First Search (BFS)**. Complete the following table.

Iteration	Queue	Visited Node
1	[B, C]	A
2	[C, D, E]	B
3	[D, E, F, G]	C
4	[E, F, G]	D
5	[F, G]	E
6	[G]	F
7	[ ] (empty)	G

Questions:

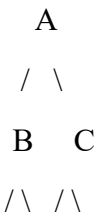
1. Write the BFS traversal order.
2. Show the queue contents after each iteration.

Answer:

1. BFS traversal order: A → B → C → D → E → F → G (level by level: A | B, C | D, E, F, G). BFS uses a FIFO queue: remove the front node, mark it visited, then add its unvisited children (left to right) at the back.

2. Queue contents after each iteration (front on the left): 1) [B, C] 2) [C, D, E] 3) [D, E, F, G] 4) [E, F, G] 5) [F, G] 6) [G] 7) [ ] (empty).

2. Depth-First Search (DFS) Using the same graph,



Iteration	Stack	Visited Node
1		
2		
3		
4		
5		
6		
7		

D E F G

Perform **Depth-First Search (DFS)** starting from **A**.

Complete the table.

Answer:

DFS uses a LIFO stack. Pop the top node, mark it visited, and push its children so that the left child is on top (shown below with the top of the stack on the left, after each iteration).

Iteration	Stack (top on left)	Visited Node
1	[B, C]	A
2	[D, E, C]	B
3	[E, C]	D
4	[C]	E
5	[F, G]	C
6	[G]	F
7	[ ] (empty)	G

DFS traversal order: A → B → D → E → C → F → G. (DFS goes down the left branch to its leaf, backtracks, then explores the right branch, unlike BFS, which goes level by level.)

3. **Initial State :**
   1 3 6
   5 0 2
   4 7 8

Goal State :
1 2 3
4 5 6
7 8 0

Task

Trace **Breadth-First Search (BFS)** until the goal state is reached.

Fill in the table.

Iteration	Current State	Queue Before Expansion	Queue After Expansion
1	S0 = 1 3 6 / 5 0 2 / 4 7 8	[S0]	[S1, S2, S3, S4]
2	S1 = 1 0 6 / 5 3 2 / 4 7 8	[S1, S2, S3, S4]	[S2, S3, S4, S5, S6]
3	S2 = 1 3 6 / 5 7 2 / 4 0 8	[S2, S3, S4, S5, S6]	[S3, S4, S5, S6, S7, S8]
4	S3 = 1 3 6 / 0 5 2 / 4 7 8	[S3, S4, S5, S6, S7, S8]	[S4, S5, S6, S7, S8, S9, S10]
5	S4 = 1 3 6 / 5 2 0 / 4 7 8	[S4, S5, S6, S7, S8, S9, S10]	[S5, S6, S7, S8, S9, S10, S11, S12]

Questions

1. Which node is expanded in each iteration?

Answer:

Assumptions: blank moves are tried in the order Up, Down, Left, Right; a state already generated is not added to the queue again. States are written row by row (rows separated by "/", 0 = blank). The tracing table above gives the first five iterations: S0 (initial) → S1 → S2 → S3 → S4, i.e. the nodes are expanded in the order they entered the queue (FIFO). New states: S1 = 1 0 6 / 5 3 2 / 4 7 8 (blank up), S2 = 1 3 6 / 5 7 2 / 4 0 8 (blank down), S3 = 1 3 6 / 0 5 2 / 4 7 8 (blank left), S4 = 1 3 6 / 5 2 0 / 4 7 8 (blank right). Complete BFS continues in the same way, level by level (depth 0, 1, 2 …), until the goal at depth 8 is expanded.

2. How many states are generated in total?

Answer:

After the first 5 iterations, 13 states have been generated (1 initial + 4 + 2 + 2 + 2 + 2). Running BFS to the end (goal tested when it is expanded, repeated states discarded) takes **311 expansions** and generates **499 states** (500 including the initial state), of which 189 are still waiting in the queue when the goal is reached.

3. Write the complete solution path from the initial state to the goal state.

Answer:

Shortest solution = 8 moves (blank moves): Right, Up, Left, Down, Left, Down, Right, Right.

- Start: 1 3 6 / 5 0 2 / 4 7 8
- Right: 1 3 6 / 5 2 0 / 4 7 8
- Up: 1 3 0 / 5 2 6 / 4 7 8
- Left: 1 0 3 / 5 2 6 / 4 7 8
- Down: 1 2 3 / 5 0 6 / 4 7 8
- Left: 1 2 3 / 0 5 6 / 4 7 8
- Down: 1 2 3 / 4 5 6 / 0 7 8
- Right: 1 2 3 / 4 5 6 / 7 0 8
- Right: 1 2 3 / 4 5 6 / 7 8 0

4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

Answer:

BFS expands nodes in order of depth: all states at depth d are expanded before any state at depth d + 1. In the 8-puzzle every move has the same cost (1), so path cost equals depth. The first time BFS reaches the goal is therefore at the smallest possible depth, and no shorter solution can exist, because every shallower state has already been examined. BFS is also complete (the branching factor is finite, at most 4) so it always finds a solution when one exists. The price is memory and time of O(b^d) (here 499 states already for depth 8).

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	**20**	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
State Expansion and Dry Run Execution	**30**	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	**20**	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	**20**	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Presentation and Logical Reasoning	**10**	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.